

Aclaración fundamental sobre el objetivo del trabajo

Todo el diseño, la documentación y las decisiones generadas en este proyecto se utilizarán posteriormente para desarrollar la aplicación real mediante herramientas de inteligencia artificial.

Por lo tanto, no quiero únicamente una propuesta visual ni una colección de mockups.

Quiero que el resultado funcione como un handoff completo, preciso y suficientemente detallado para que otra IA pueda utilizarlo como especificación de desarrollo sin tener que reinterpretar decisiones importantes.

Cada pantalla, componente, interacción, fórmula, entidad, servicio y regla de negocio debe quedar documentada de manera explícita.

Cuando exista una decisión ambigua:

1. Tomar una decisión razonable.
2. Documentar la decisión.
3. Explicar brevemente el motivo.
4. Mantenerla consistente en todo el producto.

No dejar funcionalidades representadas únicamente de forma visual. Cada elemento interactivo debe indicar:

- Qué acción ejecuta.
- Qué datos necesita.
- Qué validaciones aplica.
- Qué servicio consume.
- Qué estado modifica.
- Qué ocurre si la operación funciona.
- Qué ocurre si falla.
- Qué permisos requiere.
- Qué eventos analíticos genera.
- Qué comportamiento tiene sin conexión.

El handoff debe poder ser usado posteriormente para implementar la aplicación en Flutter o React Native.

Registro de ejercicio y actividad física

La aplicación también debe permitir registrar ejercicio, actividad física y gasto energético estimado.

El ejercicio debe estar integrado al resumen diario, historial, progreso y sistema de objetivos.

No presentar las calorías quemadas como mediciones exactas. Mostrar siempre que son estimaciones, especialmente cuando fueron calculadas manualmente o mediante fórmulas MET.

Formas de agregar ejercicio

Desde el botón principal de agregar, incluir:

- Agregar comida.
- Sacar foto de una comida.
- Escanear alimento.
- Agregar ejercicio.
- Registrar peso.
- Registrar medida corporal.

Al seleccionar “Agregar ejercicio”, ofrecer:

- Buscar actividad.
- Actividad reciente.
- Actividad favorita.
- Crear actividad manual.
- Importar desde un dispositivo o plataforma de salud.
- Registrar entrenamiento de fuerza.
- Registrar caminata o carrera.
- Registrar actividad por duración.
- Registrar actividad por distancia.

Tipos de ejercicio

Contemplar inicialmente:

- Caminata.
- Carrera.
- Ciclismo.
- Natación.
- Entrenamiento de fuerza.
- Entrenamiento funcional.
- Yoga.
- Pilates.
- Deportes.
- Baile.
- Senderismo.
- Elíptico.
- Remo.
- Escaleras.
- Actividad personalizada.

Cada actividad debe poder guardar:

- Nombre.
- Categoría.
- Fecha.
- Hora de inicio.
- Duración.
- Intensidad.
- Distancia opcional.
- Cantidad de pasos opcional.
- Frecuencia cardíaca promedio opcional.
- Frecuencia cardíaca máxima opcional.
- Calorías estimadas.
- Método de estimación.
- Fuente de información.
- Notas.
- Foto opcional.
- Dispositivo de origen opcional.
- Identificador externo opcional.

Registro rápido de ejercicio

Crear una pantalla optimizada para registrar una actividad en pocos segundos.

Campos principales:

- Tipo de actividad.
- Duración.
- Intensidad.
- Fecha y hora.
- Calorías estimadas.

Campos opcionales:

- Distancia.
- Pasos.
- Frecuencia cardíaca.
- Notas.

Al cambiar el tipo, duración, intensidad o peso corporal, recalcular el gasto estimado.

Permitir que el usuario sobrescriba manualmente las calorías calculadas.

Cuando el usuario sobrescriba el valor, guardar:

- Valor originalmente calculado.
- Valor corregido.

- Motivo opcional.
- Método final utilizado.

Entrenamiento de fuerza

Permitir registrar una sesión simple de entrenamiento de fuerza.

En el MVP debe ser posible guardar:

- Nombre del entrenamiento.
- Duración total.
- Intensidad.
- Calorías estimadas.
- Notas.

Preparar la arquitectura para una versión posterior con:

- Ejercicios individuales.
- Series.
- Repeticiones.
- Peso utilizado.
- Tiempo de descanso.
- Volumen total.
- Récords personales.
- Rutinas reutilizables.

No es necesario convertir la primera versión en una aplicación completa de seguimiento de gimnasio, pero el modelo de datos no debe impedir agregar esta funcionalidad después.

Actividades frecuentes

Permitir:

- Guardar actividades favoritas.
- Repetir una actividad reciente.
- Duplicar una sesión anterior.
- Crear plantillas.
- Editar actividades importadas.
- Identificar actividades registradas manualmente o importadas.

Ejemplo:

Caminata rápida

30 minutos

Intensidad moderada

Gasto estimado: 180 kcal

Pantalla principal con actividad física

La pantalla de inicio debe mostrar tanto alimentación como actividad física.

La tarjeta principal debe incluir:

- Calorías consumidas.
- Objetivo base.
- Calorías de ejercicio estimadas.
- Presupuesto ajustado opcional.
- Calorías restantes.
- Balance diario.

Ejemplo:

2.100 kcal de objetivo base

1.640 kcal consumidas

240 kcal de actividad registradas

700 kcal restantes con ajuste de actividad

Agregar una configuración llamada:

“Sumar las calorías del ejercicio a mi presupuesto diario”.

Opciones:

- No sumar ejercicio.
- Sumar el 50%.
- Sumar el 75%.
- Sumar el 100%.
- Porcentaje personalizado.

Utilizar por defecto una opción conservadora, como no sumar ejercicio o sumar solamente una parte.

Explicar:

“Las calorías quemadas durante el ejercicio suelen sobreestimarse. Podés decidir cuánto de ese gasto agregar a tu presupuesto diario.”

El usuario debe poder desactivar completamente el ajuste por ejercicio.

Formas de visualizar el balance

Mostrar por separado:

- Objetivo base.

- Comida consumida.
- Actividad física.
- Ajuste aplicado.
- Balance final.

No esconder el cálculo detrás de un único número.

Fórmula visual:

objetivo ajustado = objetivo base + calorías de ejercicio aplicadas

calorías restantes = objetivo ajustado – calorías consumidas

Ejemplo cuando se aplica el 50%:

$\text{caloríasEjercicioAplicadas} = \text{caloríasEjercicioEstimadas} \times 0,50$

$\text{objetivoAjustado} = \text{objetivoBase} + \text{caloríasEjercicioAplicadas}$

Sección de actividad del día

Debajo de las comidas, incluir una sección llamada “Actividad”.

Mostrar:

- Actividades registradas.
- Duración.
- Calorías estimadas.
- Fuente.
- Método de cálculo.
- Hora.
- Estado de sincronización.

Permitir:

- Editar.
- Duplicar.
- Eliminar.
- Marcar como favorita.
- Ver detalles.
- Cambiar calorías manualmente.

Fórmulas para ejercicio

Implementar todas las fórmulas como funciones independientes, testeables y documentadas.

Estimación mediante MET

Usar la fórmula:

$$\text{caloriasPorMinuto} = \text{MET} \times 3,5 \times \text{pesoKg} \div 200$$

$$\text{caloriasActividad} = \text{caloriasPorMinuto} \times \text{duracionMinutos}$$

Forma equivalente:

$$\text{caloriasActividad} = \text{MET} \times 3,5 \times \text{pesoKg} \div 200 \times \text{duracionMinutos}$$

Ejemplo conceptual:

- Peso: 100 kg.
- Actividad: caminata moderada.
- MET: 3,5.
- Duración: 30 minutos.

No hardcodear los valores MET dentro de los componentes visuales.

Crear una tabla o catálogo configurable de actividades y valores MET.

Intensidad

Cuando una actividad tenga varias intensidades, utilizar distintos valores MET:

- Suave.
- Moderada.
- Intensa.

Cada actividad debe poder definir un valor MET por intensidad.

Ejemplo conceptual:

```
{ "activity": "walking", "intensity": "moderate", "met": 3.5 }
```

Los valores deben poder modificarse posteriormente sin actualizar la interfaz.

Gasto basado en información externa

Cuando una actividad provenga de Apple Health, Health Connect u otro proveedor:

- Guardar el valor original.
- Guardar la unidad.
- Guardar la fuente.

- Guardar el identificador externo.
- Evitar duplicados.
- No recalcular automáticamente salvo que el valor sea inválido.
- Permitir que el usuario lo edite.
- Marcar el dato como importado.

Gasto basado en frecuencia cardíaca

No implementar una fórmula casera de frecuencia cardíaca como fuente principal durante el MVP.

Dejar preparada una interfaz para utilizar:

- Calorías informadas por un wearable.
- Datos procesados por Apple Health.
- Datos procesados por Health Connect.
- Integraciones futuras con proveedores especializados.

Calorías netas

$\text{caloriasNetas} = \text{caloriasConsumidas} - \text{caloriasEjercicioAplicadas}$

Mostrar esta métrica solamente cuando sea comprensible para el usuario.

No utilizarla como única representación del balance diario.

Ajuste aplicado al objetivo

$\text{factorRecuperacionEjercicio} = \text{porcentaje configurado por el usuario}$

$\text{caloriasEjercicioAplicadas} = \text{caloriasEjercicioEstimadas} \times \text{factorRecuperacionEjercicio}$

$\text{objetivoAjustado} = \text{objetivoBase} + \text{caloriasEjercicioAplicadas}$

$\text{caloriasRestantes} = \text{objetivoAjustado} - \text{caloriasConsumidas}$

Prevención de doble conteo

El sistema debe evitar sumar dos veces una misma actividad.

Posibles duplicados:

- Actividad manual más actividad importada.
- Apple Health más Health Connect.
- Entrenamiento importado más pasos asociados.
- Dos sincronizaciones del mismo proveedor.

- Actividad editada que vuelve a importarse.

Definir una estrategia de deduplicación basada en:

- Proveedor.
- Identificador externo.
- Hora de inicio.
- Hora de finalización.
- Tipo de actividad.
- Duración.
- Calorías.
- Distancia.

Si existe una posible coincidencia, solicitar confirmación o marcarla para revisión.

Historial de actividad

En el historial diario mostrar:

- Comidas.
- Actividades.
- Peso.
- Objetivo base.
- Ajuste por ejercicio.
- Balance final.

Agregar filtros para:

- Solamente comidas.
- Solamente ejercicio.
- Actividades manuales.
- Actividades importadas.
- Tipo de actividad.
- Rango de fechas.

Cada día debe mostrar:

- Calorías consumidas.
 - Calorías de actividad.
 - Calorías aplicadas al objetivo.
 - Duración total de ejercicio.
 - Cantidad de actividades.
 - Pasos, si están disponibles.
-

Progreso de ejercicio

Agregar una sección específica dentro de Progreso.

Gráfico de actividad

Mostrar:

- Minutos de actividad por día.
- Calorías estimadas por actividad.
- Cantidad de sesiones.
- Promedio semanal.
- Comparación con la semana anterior.

Gráfico por categoría

Mostrar la distribución del tiempo entre:

- Caminata.
- Carrera.
- Fuerza.
- Ciclismo.
- Deportes.
- Otras actividades.

Tarjetas de actividad

Mostrar:

- Minutos activos durante el período.
- Sesiones realizadas.
- Promedio de duración.
- Calorías estimadas.
- Actividad más frecuente.
- Mayor duración registrada.
- Días con actividad.
- Promedio de pasos, si existe información.

Consistencia

Mostrar mensajes neutrales:

- "Registraste actividad durante 4 de los últimos 7 días."
- "Esta semana acumulaste 130 minutos de actividad."
- "Caminaste 25 minutos más que la semana anterior."

No utilizar mensajes culpabilizantes.

No castigar días de descanso.

Permitir registrar un día como descanso planificado.

Objetivos de actividad

Permitir configurar objetivos opcionales:

- Minutos activos por semana.
- Sesiones por semana.
- Pasos diarios.
- Distancia semanal.
- Entrenamientos de fuerza por semana.
- Días activos por semana.

Estos objetivos no deben ser obligatorios para utilizar la aplicación.

Mostrar el progreso de forma separada al objetivo calórico.

No convertir automáticamente el incumplimiento del objetivo de ejercicio en una reducción de calorías permitidas.

Integraciones de actividad física

Apple HealthKit

Preparar interfaces para leer, con consentimiento:

- Entrenamientos.
- Energía activa.
- Pasos.
- Distancia.
- Peso.
- Frecuencia cardíaca.
- Minutos de ejercicio.

Android Health Connect

Preparar interfaces para leer, con consentimiento:

- Exercise sessions.
- Active calories burned.
- Steps.
- Distance.
- Weight.
- Heart rate.
- Total calories burned.

Reglas de integración

- Solicitar solamente permisos necesarios.
 - Explicar por qué se solicita cada permiso.
 - Permitir utilizar la app sin conectar servicios externos.
 - Permitir desconectar una integración.
 - Permitir borrar los datos importados.
 - Guardar la última fecha de sincronización.
 - Informar errores de sincronización.
 - Evitar duplicados.
 - Ejecutar sincronización incremental.
 - No sobrescribir registros manuales sin confirmación.
-

Modelo de datos adicional

activities

- id.
- user_id.
- activity_type_id.
- custom_name opcional.
- started_at.
- ended_at opcional.
- duration_minutes.
- intensity.
- distance_meters opcional.
- steps opcional.
- average_heart_rate opcional.
- maximum_heart_rate opcional.
- estimated_calories.
- original_calories opcional.
- applied_calories.

- exercise_credit_percentage.
- estimation_method.
- source_type.
- external_source opcional.
- external_id opcional.
- notes opcional.
- photo_path opcional.
- is_favorite.
- sync_status.
- created_at.
- updated_at.
- deleted_at opcional.

activity_types

- id.
- slug.
- display_name.
- category.
- default_met.
- light_met opcional.
- moderate_met opcional.
- vigorous_met opcional.
- supports_distance.
- supports_steps.
- supports_heart_rate.
- icon_name.
- is_system.
- created_at.
- updated_at.

activity_goals

- id.
- user_id.
- goal_type.
- target_value.
- unit.
- period.
- start_date.
- end_date opcional.
- enabled.
- created_at.
- updated_at.

exercise_templates

- id.
- user_id.
- name.
- activity_type_id.
- default_duration_minutes.
- default_intensity.
- default_distance_meters opcional.
- default_notes opcional.
- created_at.
- updated_at.

health_integrations

- id.
- user_id.
- provider.
- status.
- permissions.
- last_sync_at.
- sync_cursor opcional.
- connected_at.
- disconnected_at opcional.
- created_at.
- updated_at.

sync_records

- id.
- user_id.
- provider.
- entity_type.
- external_id.
- local_entity_id.
- source_updated_at.
- synced_at.
- sync_hash.
- created_at.

Crear índices para:

- user_id.
- started_at.
- external_source y external_id.
- activity_type_id.
- deleted_at.

Crear una restricción única apropiada para impedir duplicados de registros externos.

Servicios adicionales

Agregar estas interfaces:

- ActivityService.
- ActivityCatalogService.
- ExerciseCalculationService.
- ExerciseTemplateService.
- ActivityGoalService.
- HealthIntegrationService.
- HealthSyncService.
- DuplicateDetectionService.
- PermissionService.

Ejemplo conceptual:

```
interface ExerciseCalculationService { calculateCaloriesFromMet(input: { met: number; weightKg: number; durationMinutes: number; }): number;
```

```
calculateAppliedExerciseCalories(input: { estimatedCalories: number; creditPercentage: number; }): number;
```

```
calculateAdjustedCalorieTarget(input: { baseTarget: number; appliedExerciseCalories: number; }): number; }
```

```
interface ActivityService { createActivity(input: CreateActivityInput): Promise<Activity>; updateActivity(id: string, input: UpdateActivityInput): Promise<Activity>; deleteActivity(id: string): Promise<void>; restoreActivity(id: string): Promise<void>; getActivity(id: string): Promise<Activity>; getActivitiesByDate(date: string): Promise<Activity[]>; duplicateActivity(id: string, date: string): Promise<Activity>; }
```

```
interface HealthSyncService { connect(provider: HealthProvider): Promise<void>; disconnect(provider: HealthProvider): Promise<void>; sync(provider: HealthProvider): Promise<SyncResult>; resolveDuplicate(input: DuplicateResolutionInput): Promise<void>; }
```

MVP obligatorio actualizado

La primera versión debe incluir:

1. Onboarding.
2. Perfil corporal.
3. Objetivo calórico calculado o manual.
4. Configuración del ajuste por ejercicio.
5. Inicio con resumen diario.

6. Registro manual de alimentos.
 7. Buscador de alimentos.
 8. Registro mediante foto e IA.
 9. Revisión y corrección de estimaciones de IA.
 10. Historial de comidas.
 11. Registro manual de ejercicio.
 12. Catálogo de actividades.
 13. Estimación de ejercicio mediante MET.
 14. Actividades recientes y favoritas.
 15. Resumen diario de actividad.
 16. Historial de ejercicio.
 17. Registro de peso.
 18. Gráficos semanales y mensuales.
 19. Gráficos de peso, calorías y actividad.
 20. Objetivos opcionales de actividad.
 21. Favoritos.
 22. Recientes.
 23. Perfil.
 24. Configuración.
 25. Mocks de autenticación.
 26. Mocks de persistencia.
 27. Datos simulados realistas.
 28. Manejo de errores.
 29. Estados vacíos.
 30. Estados sin conexión.
 31. Prevención básica de duplicados.
 32. Interfaces preparadas para HealthKit y Health Connect.
 33. Documentación completa para handoff de desarrollo.
-

Componentes reutilizables adicionales

Crear:

- ActivitySummaryCard.
- ActivityListItem.
- ActivityTypeSelector.
- ActivityIntensitySelector.
- DurationInput.
- DistanceInput.
- ExerciseCaloriesEstimate.
- ExerciseCreditSelector.
- ActivityHistoryChart.
- ActivityCategoryChart.
- ActivityGoalCard.
- HealthIntegrationCard.

- SyncStatusBadge.
- DataOriginBadge.
- DuplicateActivityDialog.
- ExerciseTemplateCard.
- ActiveMinutesCard.
- StepsSummaryCard.

Cada componente debe documentar:

- Props.
- Estados.
- Eventos.
- Variantes.
- Validaciones.
- Dependencias.
- Reglas de accesibilidad.
- Ejemplo de uso.
- Comportamiento en carga.
- Comportamiento en error.
- Comportamiento sin datos.

Handoff obligatorio para desarrollo

El resultado final debe incluir una carpeta conceptual o real llamada:

/docs/handoff

Dentro de ella generar documentos separados y estructurados.

1. product-requirements.md

Debe contener:

- Objetivo del producto.
- Público objetivo.
- Problemas que resuelve.
- Alcance del MVP.
- Funciones posteriores.
- Reglas de negocio.
- Limitaciones conocidas.
- Suposiciones.
- Riesgos.
- Criterios de éxito.

2. information-architecture.md

Debe contener:

- Mapa completo de navegación.
- Jerarquía de pantallas.
- Rutas.
- Modales.
- Bottom sheets.
- Navegación autenticada.
- Navegación sin autenticación.
- Deep links futuros.

3. user-flows.md

Documentar paso a paso:

- Crear cuenta.
- Completar onboarding.
- Agregar alimento manual.
- Buscar alimento.
- Analizar foto.
- Corregir análisis.
- Guardar comida.
- Agregar ejercicio.
- Importar ejercicio.
- Corregir calorías de ejercicio.
- Registrar peso.
- Consultar historial.
- Cambiar objetivo.
- Eliminar cuenta.

Cada flujo debe incluir:

- Punto de entrada.
- Pasos.
- Validaciones.
- Errores.
- Resultado final.
- Estado persistido.

4. screen-specifications.md

Crear una especificación individual para cada pantalla.

Cada pantalla debe incluir:

- Nombre.
- Ruta.
- Objetivo.
- Información mostrada.
- Acciones disponibles.
- Componentes utilizados.
- Datos requeridos.
- Estados.
- Errores.
- Validaciones.
- Navegación de entrada.
- Navegación de salida.
- Eventos analíticos.
- Requisitos de accesibilidad.

5. component-library.md

Documentar todos los componentes:

- Nombre.
- Responsabilidad.
- Props.
- Eventos.
- Variantes.
- Estados.
- Dependencias.
- Ejemplos.
- Reglas responsive.
- Reglas de accesibilidad.

6. design-tokens.md

Definir:

- Colores.
- Tipografías.
- Espaciados.
- Radios.
- Sombras.
- Tamaños de íconos.
- Breakpoints.
- Duraciones de animación.
- Elevaciones.
- Estados de interacción.

Entregar los tokens también en formato compatible con código, por ejemplo JSON o TypeScript.

7. data-model.md

Documentar:

- Tablas.
- Columnas.
- Tipos.
- Relaciones.
- Índices.
- Restricciones.
- Campos opcionales.
- Soft deletes.
- Estrategia de auditoría.
- Estrategia de migraciones.

Incluir un diagrama entidad-relación.

8. supabase-plan.md

Documentar:

- Tablas.
- Migraciones.
- Buckets.
- Rutas de archivos.
- Políticas RLS.
- Edge Functions.
- Secretos.
- Triggers.
- Índices.
- Jobs.
- Rate limiting.
- Estrategia de backups.
- Eliminación de cuenta.
- Exportación de datos.

Incluir ejemplos conceptuales de políticas RLS y SQL.

9. api-contracts.md

Definir contratos para:

- Buscar alimentos.
- Obtener detalle nutricional.
- Analizar foto.

- Crear comida.
- Crear actividad.
- Sincronizar HealthKit.
- Sincronizar Health Connect.
- Registrar peso.
- Obtener resumen diario.
- Obtener progreso.

Cada endpoint o función debe incluir:

- Nombre.
- Método.
- Entrada.
- Salida.
- Errores.
- Autenticación.
- Idempotencia.
- Paginación.
- Ejemplo JSON.

10. types-and-interfaces.ts

Generar tipos TypeScript para:

- UserProfile.
- Goal.
- Food.
- Meal.
- MealItem.
- Activity.
- ActivityType.
- WeightLog.
- AIAnalysis.
- DailySummary.
- ProgressSummary.
- HealthIntegration.
- SyncRecord.
- API responses.
- API errors.
- Form inputs.
- Service interfaces.

Aunque la app final pueda desarrollarse en Flutter, estos tipos funcionarán como contrato de referencia.

11. calculation-rules.md

Documentar todas las fórmulas:

- BMR.
- TDEE.
- Déficit.
- Superávit.
- Calorías por gramos.
- Calorías por macros.
- Promedios móviles.
- Adherencia.
- IMC.
- MET.
- Ajuste por ejercicio.
- Balance diario.
- Tendencia de peso.

Para cada fórmula incluir:

- Variables.
- Unidades.
- Redondeo.
- Casos inválidos.
- Ejemplos.
- Tests esperados.

12. external-integrations.md

Documentar:

- Gemini.
- USDA FoodData Central.
- Open Food Facts.
- Supabase.
- Apple HealthKit.
- Android Health Connect.

Para cada integración incluir:

- Finalidad.
- Autenticación.
- Variables de entorno.
- Límites.
- Errores.
- Reintentos.
- Cache.

- Privacidad.
- Mocks.
- Estrategia de reemplazo.

13. state-management.md

Definir:

- Estado global.
- Estado local.
- Estado del servidor.
- Cache.
- Persistencia offline.
- Invalidación.
- Optimistic updates.
- Sincronización.
- Manejo de conflictos.

14. analytics-events.md

Definir los eventos analíticos:

- onboarding_started.
- onboarding_completed.
- meal_created.
- meal_photo_analyzed.
- ai_result_corrected.
- food_searched.
- activity_created.
- activity_imported.
- activity_calories_edited.
- weight_logged.
- goal_updated.
- integration_connected.
- integration_sync_failed.

Para cada evento incluir:

- Momento de emisión.
- Propiedades.
- Datos que nunca deben enviarse.
- Finalidad.

15. accessibility.md

Documentar:

- Navegación por lector de pantalla.
- Labels.
- Orden de foco.
- Contraste.
- Texto escalable.
- Tamaños táctiles.
- Reducción de movimiento.
- Gráficos accesibles.
- Mensajes de error.

16. test-plan.md

Crear:

- Tests unitarios.
- Tests de componentes.
- Tests de integración.
- Tests end-to-end.
- Tests de fórmulas.
- Tests offline.
- Tests de sincronización.
- Tests de permisos.
- Tests de RLS.
- Tests de eliminación de cuenta.
- Tests de duplicados.

17. acceptance-criteria.md

Cada funcionalidad del MVP debe tener criterios en formato Given, When, Then.

Ejemplo:

Given que el usuario pesa 100 kg
And selecciona caminata moderada con MET 3,5
When registra una duración de 30 minutos
Then la aplicación calcula aproximadamente 184 kcal
And muestra que el resultado es estimado
And permite modificarlo antes de guardar

18. implementation-roadmap.md

Dividir el desarrollo en fases:

- Fundación.
- Diseño y navegación.
- Autenticación.
- Perfil y objetivos.
- Registro de comidas.
- Integración nutricional.
- Análisis mediante IA.
- Registro de actividad.
- Historial.
- Progreso.
- Integraciones de salud.
- Offline.
- Seguridad.
- Testing.
- Publicación.

Cada fase debe incluir:

- Dependencias.
- Entregables.
- Riesgos.
- Definición de terminado.

19. project-structure.md

Proponer una estructura de carpetas para:

- Flutter.
- React Native.

Separar:

- Presentación.
- Dominio.
- Datos.
- Servicios.
- Modelos.
- Repositorios.
- Casos de uso.
- Componentes.
- Utilidades.
- Tests.
- Configuración.

20. environment-variables.md

Definir variables necesarias, sin valores reales:

- SUPABASE_URL.
- SUPABASE_ANON_KEY.
- SUPABASE_SERVICE_ROLE_KEY.
- GEMINI_API_KEY.
- USDA_API_KEY.
- OPEN_FOOD_FACTS_USER_AGENT.
- SENTRY_DSN.
- APP_ENVIRONMENT.
- API_BASE_URL.

Indicar cuáles pertenecen:

- Al cliente.
- Al backend.
- A Edge Functions.
- Al pipeline de CI/CD.

Nunca colocar secretos privados en la aplicación cliente.

Entregables finales actualizados

Entregar:

1. Aplicación móvil navegable.
2. Mockups de alta fidelidad.
3. Flujos funcionales.
4. Estados completos.
5. Registro de comidas.
6. Registro mediante IA.
7. Registro de ejercicio.
8. Cálculo mediante MET.
9. Historial.
10. Registro de peso.
11. Gráficos.
12. Objetivos.
13. Modo claro.
14. Modo oscuro.
15. Componentes reutilizables.
16. Design tokens.
17. Datos simulados.
18. Tipos y contratos.

19. Modelo de datos.
20. Diagrama de base de datos.
21. Interfaces de servicios.
22. Contratos de APIs.
23. Especificación de Supabase.
24. Esquema de salida de Gemini.
25. Estrategia offline.
26. Estrategia de sincronización.
27. Estrategia de seguridad.
28. Estrategia de testing.
29. Criterios de aceptación.
30. Roadmap de implementación.
31. Estructura recomendada del proyecto.
32. Documentación completa dentro de /docs/handoff.
33. Lista de decisiones tomadas.
34. Lista de funcionalidades simuladas.
35. Lista de elementos todavía no implementados.
36. Lista de pasos exactos para comenzar el desarrollo.

El resultado debe ser suficientemente completo para entregar el repositorio o la documentación a otra IA y pedirle:

“Desarrollá la aplicación siguiendo exactamente este handoff.”

No asumir que la IA desarrolladora tendrá acceso al contexto original de esta conversación.

Toda la información necesaria debe existir dentro del proyecto y su documentación.

No utilizar expresiones ambiguas como:

- “Agregar lógica correspondiente”.
- “Conectar al backend”.
- “Implementar según necesidad”.
- “Manejar errores”.
- “Crear los componentes necesarios”.

En lugar de eso, especificar exactamente:

- Qué lógica.
- Qué backend.
- Qué endpoint.
- Qué errores.
- Qué componentes.
- Qué datos.
- Qué estados.
- Qué validaciones.
- Qué resultado esperado.

No detenerse a solicitar confirmaciones menores. Tomar decisiones razonables, documentarlas y completar el handoff.